



Trabajo Práctico: Testing Automático

Objetivo:

El objetivo de esta actividad es incorporar testing automático sobre una aplicación ya existente, utilizando herramientas modernas de testing utilizadas actualmente en entornos profesionales de desarrollo frontend.

No se debe crear una aplicación nueva. Cada grupo deberá trabajar sobre el mismo repositorio utilizado en el TP 2 de React, agregando la configuración necesaria y los archivos de test correspondientes.

Requerimientos:

1) Instalar y configurar el entorno de testing

Configurar el proyecto React para soportar tests automáticos utilizando:

- Vitest
- React Testing Library
- jest-dom
- user-event

El proyecto debe poder ejecutar tests mediante:

```
npm run test  
npm run test:run
```

2) Configurar el proyecto

Realizar la configuración necesaria para:

- Configurar *jsdom* como entorno de testing.
- Crear un archivo de setup para importar *@testing-library/jest-dom*.
- Agregar los scripts correspondientes en *package.json*.

3) Crear tests automáticos

La aplicación deberá incorporar tests automáticos sobre los componentes y funcionalidades desarrolladas en el TP 2.

Se espera que cada componente reutilizable o componente con lógica propia tenga al



menos un archivo de test asociado.

Buenas prácticas

Los tests deben:

- Tener nombres descriptivos
- Ser legibles
- Ser independientes entre sí
- Validar comportamiento observable para el usuario
- Simular acciones reales del usuario
- Evitar duplicación innecesaria

Evitar

- Testear detalles internos de implementación
- Testear variables internas
- Testear *useState*
- Duplicar lógica del componente dentro del test
- Crear tests triviales sin validaciones relevantes

Ejemplo de nomenclatura esperada

```
describe("MovieCard", () => {  
  it("muestra correctamente el título de la película", () => {  
  
  });  
  
  it("llama a onDelete cuando el usuario hace click en eliminar", () => {  
  
  });  
});
```

README

Actualizar el archivo README.md agregando:

- Librerías instaladas para testing
 - Cómo ejecutar los tests
-



Modalidad de entrega:

La entrega deberá realizarse sobre el mismo repositorio utilizado en el TP 2.

El proyecto deberá poder ejecutarse correctamente utilizando:

```
npm install  
npm run test
```

Todos los tests deben ejecutarse correctamente sin errores.

Criterios de evaluación:

Se tendrá en cuenta:

- Calidad de los tests
- Cobertura de funcionalidades relevantes
- Buenas prácticas de testing
- Organización del código
- Claridad y legibilidad
- Uso correcto de React Testing Library

Bonus opcional

No obligatorio.

Algunas ideas posibles para continuar aprendiendo y mejorar más el testing:

- Testear localStorage
- Mockear APIs
- Testear navegación
- Testear hooks personalizados
- Incorporar coverage
- Incorporar CI automático para ejecutar tests en Pull Requests



Documentación oficial recomendada

Durante el desarrollo del trabajo práctico se recomienda consultar la documentación oficial de las herramientas utilizadas.

Vitest

[Documentación oficial de Vitest](#)

React Testing Library

[Documentación oficial de React Testing Library](#)

jest-dom

[Documentación oficial de jest-dom](#)

user-event

[Documentación oficial de user-event](#)

Estructura de Directorios:

Estructura de archivos y carpetas recomendada:

```
-Pages
  --Home
    --Home.jsx
    --Home.module.css
    --Home.test.js
  --Details
    --Details.jsx
    --Details.module.css
    --Details.test.js

-Components
  --Button
    --Button.jsx
    --Button.module.css
    --Button.test.js
```



Facultad de Informática
Programación Web Avanzada



```
--OtroComponente
  --OtroComponente.jsx
  --OtroComponente.module.css
  --OtroComponente.test.js
```

```
-services
  getAllXYZ.js
  getAllXYZ.test.js
  getXYZBYId.js
  getXYZBYId.test.js
```